Automated Software Engineering (2025) 32:34
https://doi.org/10.1007/s10515-025-00504-w

Ladle: a method for unsupervised anomaly detection across log types

Juha Mylläri[1] · **Tatu Aalto**[2] · **Jukka K. Nurminen**[1]

Received: 19 September 2024 / Accepted: 28 February 2025 / Published online: 24 March 2025

Abstract

Log files can help detect and diagnose erroneous software behaviour, but their utility is limited by the ability of users and developers to sift through large amounts of text. Unsupervised machine learning tools have been developed to automatically find anomalies in logs, but they are usually not designed for situations where a large number of log streams or log files, each with its own characteristics, need to be analyzed and their anomaly scores compared. We propose Ladle, an accurate unsupervised anomaly detection and localization method that can simultaneously learn the characteristics of hundreds of log types and determine which log entries are the most anomalous across these log types. Ladle uses a sentence transformer (a large language model) to embed short overlapping segments of log files and compares new, potentially anomalous, log segments against a collection of reference data. The result of the comparison is re-centered by subtracting a baseline score indicating how much variation tends to occur in each log type, making anomaly scores comparable across log types. Ladle is designed to adapt to data drift and is updated by adding new reference data without the need to retrain the sentence transformer. We demonstrate the accuracy of Ladle on a real-world dataset consisting of logs produced by an endpoint protection platform test suite. We also compare Ladle's performance on the dataset to that of a state-of-the-art method for single-log anomaly detection, showing that the latter is inadequate for the multi-log task.

Keywords Software logs · Anomaly detection · Anomaly localization · Software testing · Unsupervised learning · Text embedding

This work was partly funded by local authorities ("Business Finland") under grant agreements 20219 IML4E and 23004 ELFMo of the ITEA4 programme.

✉ Juha Mylläri
juha.myllari@helsinki.fi

[1] University of Helsinki, Helsinki, Finland

[2] WithSecure, Helsinki, Finland

1 Introduction

Anomalous or atypical entries in automatically generated logs can help detect or diagnose erroneous software behaviour. Unsupervised machine learning tools can help find such anomalies from large quantities of log text even if labelled training data are not available. However, most recently published tools (Le and Zhang 2021; Duo et al. 2021; Pan et al. 2023) for anomaly detection or localization in logs are designed to monitor a single stream of log entries. If the log outputs contain additional structure, these tools may leave valuable information on the table. A particularly important form of such structure is when different software components output their log entries into separate files. We will consider each such file an instance of a log type. In this paper, we propose an accurate method, Ladle (Log anomaly detector and localizer), for unsupervised anomaly detection and localization in log data consisting of multiple log types.

The presence of a large number (tens or hundreds) of log types presents a challenge. If a separate machine learning model is trained for each log type, how should the models be calibrated to produce anomaly scores that are comparable between log types? If, on the other hand, a single model is used for all log types, can it learn the characteristics of each log type without requiring an inordinate amount of training data? Ladle avoids these problems by using a single large language model (LLM), specifically a sentence transformer (Reimers et al. 2019), to embed all log entries and then comparing the embeddings with reference embeddings for each log type. The sentence transformer gives Ladle the ability to assess the proximity of new log entries to known log entries in meaning, not merely in string similarity.

The anomaly scores assigned to log entries should be comparable between log types. Even in non-anomalous situations, some log files will have much variation between different runs of the log-producing software, while others will have almost none. If this is not taken into account, random fluctuations in highly variable log files will likely eclipse actual anomalies in relatively unchanging log files. Ladle handles the problem of differential variation by re-centring anomaly scores assigned to log entries by a baseline score that indicates how much variation typically occurs in that log type.

A log anomaly detection system should be able to adapt quickly to data drift. Updates to the software product itself or changes in the environment in which it is run (including other software, networking conditions, etc.) may result in changes to the log contents. In most cases, such changes are likely to be benign and not associated with software misbehaviour. While it is reasonable to initially label these changes as anomalous, the anomaly detection system should start treating them as normal once sufficient new reference data have accumulated. Ladle is updated by adding new entries to the reference data; there is no need to retrain the LLM. Furthermore, Ladle includes a human-interpretable parameter that controls how quickly it adapts to novel features in the data.

We demonstrate the accuracy of Ladle using data from an actual industry use case. The use case concerns the software test suite of an endpoint protection

platform (EPP),[1] a single run of which yields around 170 separate log files. The dataset consists of 211 such runs. Each log file is generated by a different software component or records a distinct process; we therefore consider them to represent different log types. The logs exhibit within-type homogeneity and between-type heterogeneity; i.e. each log file tends to be similar to the corresponding log in previous test runs but dissimilar to the other logs in the same test run. We show that Ladle is highly accurate in detecting artificially generated anomalies in the data. Furthermore, we test a state-of-the-art unsupervised log anomaly detection method (Zhang et al. 2022) on the same dataset and demonstrate its inability to correctly detect anomalies over different log types.

The main contributions of this article are as follows.

1. We describe a category of real-world log anomaly detection and localization problems for which existing methods are not well suited. Such problems are characterized by the presence of multiple log types, within-type homogeneity, between-type heterogeneity, and data drift.
2. We propose Ladle, an accurate unsupervised anomaly detection and localization method for logs exhibiting these properties. We make the code for Ladle publicly available.
3. We test Ladle on a real-world log dataset by introducing artificial anomalies, demonstrating the method's accuracy. Further, we compare the performance of Ladle with that of a state-of-the-art unsupervised log anomaly detection method, demonstrating that the latter is inadequate for the problem.

2 Defining the Problem

2.1 Anomaly detection across log types

We study the problem of unsupervised anomaly detection and localization in log files where the data exhibit a particular set of properties:

1. **Distinct log types:** The data can be divided into a large number (tens or hundreds) of log types. For example, if different software components output their log entries into separate files, each such log file constitutes a log type.
2. **Within-type homogeneity:** When the software that produces the logs is run on multiple occasions, the log entries for each log type tend to resemble the entries (of the same log type) in previous runs.
3. **Between-type heterogeneity:** The log types are dissimilar; i.e. the log contents tend to differ much more between log types than between software runs for the same log type.
4. **Data drift:** What constitutes a normal log for a given log type may change over time.

[1] An EPP is a software solution that is intended to protect a device against malware, intrusion etc.

For simplicity, we will assume that each log type is represented by a unique file name or path that remains constant between runs. Concretely, we can think of the log files as having names like `database.log`, `file_io.log`, `model.log`, `networking.log`, `backend.log` etc. depending on the component or process that produces them. (Of course, if the software produces hundreds of log files, not all are likely to be named in quite such an intuitive and self-explanatory way.) Each log file name then corresponds to a log type. It is not required that all log types are present in all runs.

Suppose that we have a collection of log data from multiple runs where no errors occurred. These constitute our reference data. Then, given a new set of log files corresponding to a single run, our aim is to assign to each log entry an anomaly score such that the most anomalous entries in the set receive the highest anomaly scores. Thus, if software misbehaviour has occurred, its cause may be probed by studying the most anomalous entries and entries near them.

A closely related task would be to monitor anomaly scores in order to detect software misbehaviour in the first place. Although our investigation focuses on the former task, our method can also be applied to the latter. This would be done by setting a threshold score such that if the anomaly score of any entry exceeds the threshold, the user is notified.

We assume that only normal (i.e. non-anomalous) log data are available for the fitting of the model. Examples of anomalies are often rare, and annotating log data is time-consuming and costly. Furthermore, even if anomalous log entries are available, they may not be representative of the kinds of anomalies the model will encounter in actual use.

In the anomaly detection and localization literature, machine learning methods that are trained on normal data are, by convention, called unsupervised methods. Strictly speaking, they are supervised methods in the sense that the training data points have an implicit label, i.e., 'normal'. From this viewpoint, a truly unsupervised method, if trained at all, would be trained on unlabeled data that might contain anomalies. However, in this article, we follow the typical convention.

2.2 On the insufficiency of existing methods

Existing methods for unsupervised log anomaly detection are designed to find anomalies in a single stream of log entries (illustrated in Fig. 1). It is, of course, possible in principle to apply a single-stream anomaly detection method to a case with multiple log streams; one can, e.g., simply concatenate the various log streams into a single one (Fig. 2). A closely related approach would be to interleave the log streams, ordering the entries by their timestamps. This would be particularly natural in an online anomaly detection setting.

Regardless of the order in which the various log streams are combined, a question arises: if the log entries are known to represent different log types, why would we not avail ourselves of this information? Presumably the probability distribution of the combined stream of log entries is a more complicated entity to model than the probability distribution of log entries in a single log type.

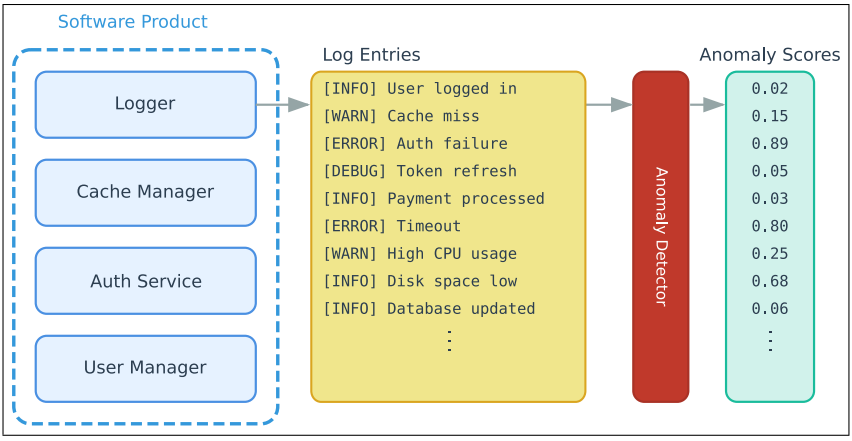


Fig. 1 Anomaly detection in a single stream of log entries. The software product produces a single stream of log entries, and the anomaly detection model assigns an anomaly score to each entry. Existing methods are well suited for this task

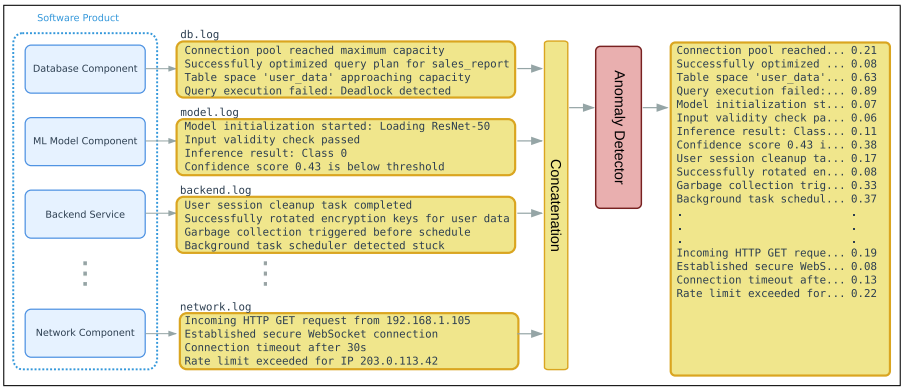


Fig. 2 Applying a traditional anomaly detection method to the concatenated outputs of various log-producing software components. We argue that this approach is not optimal and present Ladle as a better alternative

Even so, if existing methods work sufficiently well in a multi-log-type setting, there is no need for a new method designed specifically for that setting. To determine whether this is the case, we applied a state-of-the-art single-stream log anomaly detection method to our data, processing the logs belonging to each log type sequentially.[2] The experiment is described in more detail in Sect. 5.4. We found the accuracy of this approach unsatisfactory, which suggests that a method specifically designed for the multi-log-type setting is needed.

[2] This is essentially the concatenation approach, but as the method in question processes log segments, we did not allow log segments to span two or more files.

A different approach based on existing methods would be to prepare a separate machine learning model for each log type. This approach, too, comes with its own set of challenges:

1. **The sufficiency of training data.** If there are *N* log types, each model can only make use of, on average, one *N*th of the training data.
2. **Resource requirements.** Depending on the model architecture, the training, storage, and periodic retraining of *N* models may be resource-intensive.
3. **Anomaly score calibration across log types.** The models have to be calibrated in some way to output anomaly scores that are comparable between log types.

While we do not view our method, Ladle, as an extension of existing approaches, it is instructive to consider how it meets these challenges:

1. Ladle makes efficient use of training data by separating the task of semantically understanding log entries from the task of learning each log type's characteristics. The first task is handled by a sentence transformer that is used in its pre-trained state without any fine-tuning on log data. The second task is achieved by storing embedded log segments from the training data.
2. Ladle uses the same sentence transformer for all log types. The method is refit by embedding and storing new log segments.
3. Ladle sets aside a part of its training data to function as a validation set. The validation set is used to determine a baseline of raw anomaly scores that tend to occur in each log type. Subtracting the log-type-specific baseline produces anomaly scores that are comparable between log types.

3 Related work

The application of machine learning to anomaly detection in log files far predates the invention of transformer-based LLMs. As discussed in He et al. (2016) and Yadav et al. (2020), machine learning models applied to supervised log anomaly detection include decision trees, logistic regression, support vector machines, as well as various neural networks such as Long Short-Term Memory (LSTM) models and recurrent neural networks (RNN). Unsupervised methods applied to log anomaly detection before transformers include autoencoders, principal component analysis (PCA) and clustering.

Due to their excellent ability to process text data, transformer models, especially BERT (Devlin et al. 2019), have been applied to log anomaly detection since their introduction. In the LogBERT (Duo et al. 2021) method, log entries are first transformed into a simpler textual form, called a log key, using a log parser. A transformer model is then trained on sequences of log keys. In inference, the transformer is tasked with predicting partially masked log sequences. If the correct log key is not within the top N predicted log keys (for some fixed N) for that mask, the corresponding log entry is considered anomalous.

NeuralLog (Le and Zhang 2021) is a supervised learning method that omits log parsing and embeds log entries directly using BERT. The embeddings are augmented with positional information using a sinusoidal position encoding scheme. A transformer-based neural network is then used to classify sequences of log entries as normal or anomalous.

Content-Aware Transformer or CAT (Zhang et al. 2022) is an unsupervised method that embeds event sequences with a transformer model. The model learns to embed normal event sequences close to each other so that in inference, any sequence whose embedding is dissimilar to normal event sequences is labelled as anomalous. CAT shows strong accuracy scores on commonly used benchmark datasets and is, to our knowledge, currently the best-performing unsupervised log anomaly detection method whose source code is freely available.

In the unsupervised MLAD (Zang et al. 2024) method, log entries are parsed using templates, and sequences of template keys are fed into a transformer model. A Gaussian Mixture Model (GMM) is used to assess the proximity of the transformer's high-dimensional vector representations of the inputs. The test results reported for MLAD exceed those of CAT. However, the code for MLAD is not publicly available and we were not able to contact the authors to request it.

Recently, retrieval-augmented generation (RAG) (Lewis et al. 2021) has also been applied to anomaly detection in logs. In the RAGLog (Pan et al. 2023) method, a vector database is populated with normal log entry embeddings. In inference, each log entry to be examined is embedded and the vector database is queried for similar entries. Using the retrieved entries as context, an LLM assesses the normality of the test entry.

Of the methods discussed above, LogBERT, CAT, MLAD, and RAGLog are - or can be - unsupervised, and are therefore to some extent applicable to our problem. However, they are not explicitly designed for anomaly detection across log types.

Numerous log datasets are available for the development and evaluation of anomaly detection and localization systems; a particularly large collection of such datasets is LogHub (Zhu et al. 2023). Unfortunately, we have not found any publicly available datasets with the properties that we study, i.e. a large number of distinct log types exhibiting within-type homogeneity, between-type heterogeneity, and data drift.

4 Method

An overview of the structure of our method, Ladle, is presented in Fig. 3, and each part of the method will be discussed in more detail later in this section. First, the normal (non-anomalous) data available for fitting the method are split randomly into a reference set and a validation set.[3] The split is performed so that every run is assigned in its entirety to either the reference set or the validation set. All logs then undergo text preprocessing with regular expressions to eliminate highly variable substrings such as timestamps. However, no parsing is performed. The preprocessed

[3] We used a 90 : 10 split.

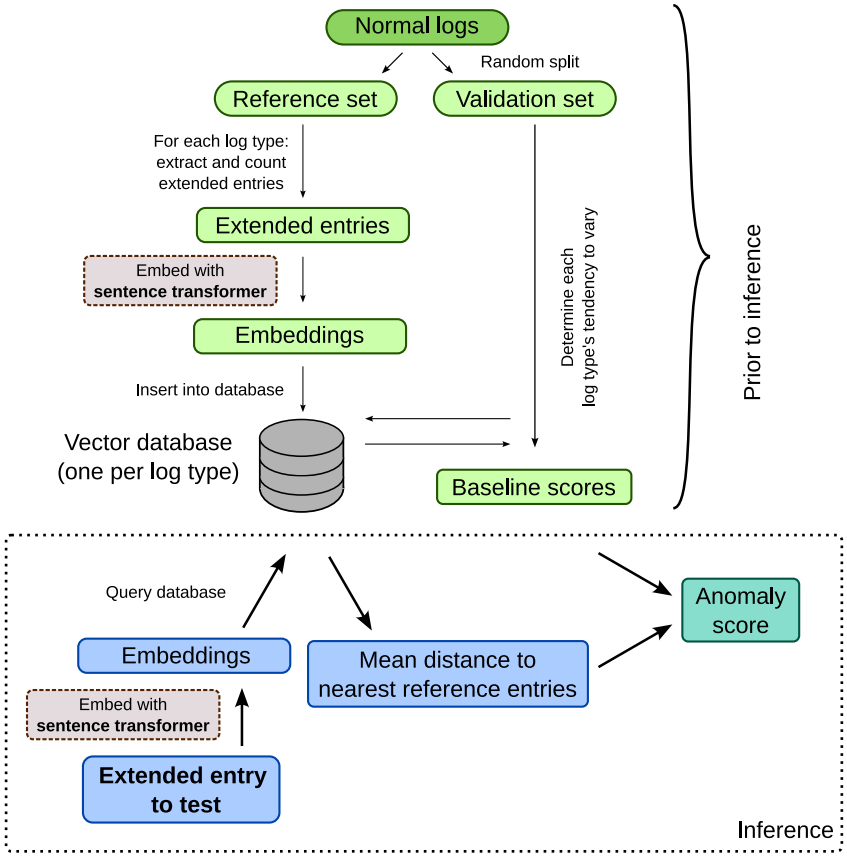


Fig. 3 An overview of our method, Ladle. A log to be tested is transformed into a sequence of extended entries, which are embedded using a sentence transformer. For each extended entry, a vector database is queried for the most similar reference embeddings. A log-type specific baseline score is subtracted from the mean distance to the nearest reference embeddings, yielding the final anomaly score for the corresponding log entry. Bold arrows indicate the flow of data during inference

log entries are transformed into overlapping segments called extended entries, each consisting of the concatenated contents of at most w consecutive log entries, for a window size w which the user provides as a hyperparameter. The extended entries are embedded, i.e. given a vector representation, using an LLM, namely a sentence transformer (Reimers et al. 2019). By embedding extended entries with a sentence transformer, Ladle is able to assess how close a test entry is to a reference entry in meaning, not merely in string similarity. All extended entries in the reference set are stored alongside their embeddings, and the embeddings are fed into a log-type-specific vector database.

The validation set is used to determine how much variation typically occurs in each log type. For each extended entry in the validation set, the vector database containing the embeddings of all extended entries in the reference set for that log type

is queried. This is to determine how distant the extended entry is from its nearest counterparts in the reference set. These distances provide an estimate of how much variation tends to occur in each log type. This information is encapsulated in a log-type-specific baseline score.

In inference, each log file goes through the steps described above: preprocessing, transformation into extended entries, and embedding of the extended entries. For each extended entry, the vector database of the corresponding log type is queried to find the extended entry's mean distance to its nearest counterparts in the reference set. The log-type-specific baseline score is then subtracted from this mean distance to account for the amount of variation that tends to occur in that log type.

Ladle is updated by adding new logs to the reference data and, if necessary, removing entries from older logs. The new reference logs are preprocessed and embedded in the same way as the original data.

We will now lay out each stage of the Ladle method in detail.

4.1 Log type assignment

Ladle assumes that each log type is uniquely identified by its file path. The path must remain constant between runs, but it is not necessary for all log types to be present in all runs. For Ladle, the set of log types is simply the set of (unique) file paths in the reference and test data.

As discussed in Sect. 2, a meaningful partition of the data into log types is one where log types exhibit within-type homogeneity and between-type heterogeneity, i.e. log files of the same type tend to have similar contents from one run to another, whereas log files of different types tend to have dissimilar contents. Whether the user needs to intervene in the assignment of log types depends on how the log-producing software organizes its outputs. We can distinguish 3 typical scenarios:

1. **Same log file names across runs.** Each software component or process outputs its log entries into a separate file, and the names of the files remain the same between runs. Each unique file name (or more precisely, file path), is treated by Ladle as a log type, and thus no intervention by the user is needed.
2. **Consistent file names with a changing element.** This is the same scenario as the first except that the file names have a timestamp or a similar changing element. The user should write a script to eliminate the changing element so that the file corresponding to a given log type has the same name in every run.
3. **No log-type specific naming.** In some cases there is no simple file renaming scheme that produces a meaningful partition of the log data into log types. For example, it may be the case that all log entries are concatenated into a single file but contain some feature (e.g. a prefix) that makes it possible to partition them according to their origin. In such cases, the user must decide what the log types are and how to assort the log entries into them.

```
Binary version: VERSION, compilation time: DATE TIME
Hotfix::Framework::PrepareTempFolder: Temporary hotfix directory: 'FILEPATH'
Downloader::DownloadTextFileToMemory: Downloading from 'URL' to 'memory'. Size limit: HEX
Downloader::DownloadTextFileToMemory: Downloading 'URL' -> 'memory' complete. Size: LARGEINT
::GetConfigurationBranch: Read values for configuration 'PSB'
::GetConfigurationBranch: Found a value for configuration 'PSB'
::GetHotfixesFromProduct: Found 0 values for the version 'VERSION'
```

Fig. 4 A segment of an actual log file from the EPP dataset after preprocessing. Variable strings like URLs have been automatically replaced by a descriptive string such as 'URL'

4.2 Log preprocessing

Unlike many recent methods such as Deeplog (Du et al. 2017), CAT (Zhang et al. 2022), and MLAD (Zang et al. 2024), Ladle treats log entries as unstructured text data, i.e. the entries are not parsed. As we use a sentence transformer to capture the meaning of each extended entry into a vector embedding, we need to maintain the entries in natural-language form.

However, logs will typically contain highly variable substrings that are not helpful for the anomaly detection task. Depending on the nature of the logs, these may include, e.g., timestamps, file paths, version numbers, and IP addresses. Ladle includes an optional preprocessing step where a list of user-defined regular expressions is applied to the logs to replace variable substrings with a descriptive substring; e.g., dates might be replaced by the string 'DATE' and IP addresses might be replaced with the string 'IP'. The replacement rules can be tailored for each log type. See Fig. 4 for a preprocessed segment of a log file in the EPP log dataset.

It is up to the user to determine which variable substrings should be replaced. An identifier that is in one context mere noise may in another context be meaningful. When in doubt, the user can leave a substring not replaced and rely on the sentence transformer to produce proximal embeddings for semantically similar strings and distant embeddings for semantically different ones. However, it should be noted that the sentence transformer's parameters are fixed; it is not fine-tuned on the reference data.

If data drift necessitates a change in the replacement rules, a bump in anomaly values is likely to be seen in the log type or types affected by the change. The elevated anomaly values will then decrease as new reference data are accumulated.

4.3 Embedding log segments

An anomaly detection method that assesses the anomalousness of each log entry in isolation fails to make use of all the information present in the data. Instead, each log entry should be examined in context. This can be seen by considering the following scenarios:

1. **Unusual order of entries.** An anomalous segment of a log may consist of entries that are not unusual for that log type but appear in an atypical order. This includes

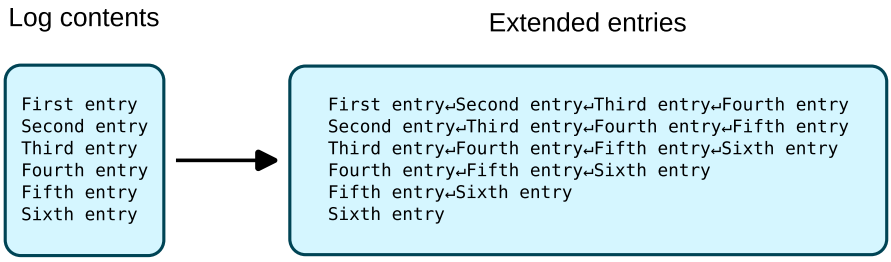


Fig. 5 Generating extended log entries with a sliding window of length 4. In the extended entries, the constituent entries are joined with newline characters. Here the log consists of six entries

situations where an entry that tends to appear early in a log file appears late or vice versa.
2. **Atypical log length.** If a log file is shorter than usual or longer than usual, this may be indicative of an anomaly.
3. **Missing entries.** An anomaly may manifest as the absence of one or more entries rather than the presence of unexpected ones.

Ladle incorporates contextual information by dividing logs into short overlapping segments called extended entries. The extended entry corresponding to entry i in a log file consists of w consecutive entries, joined with the newline character, starting from entry i. While Ladle allows w to be set freely, we used the value $w = 4$ in all our experiments for reasons discussed below.

An extended entry is created for every entry in a log. For the last $w - 1$ entries in a log file, the extended entries terminate at the end of the file. For an illustration of how the extended entries are generated, see Fig. 5.

Ladle embeds each extended entry using a sentence transformer (Reimers et al. 2019), an LLM that takes a text input and outputs an embedding. In the case of the model on which we conducted most of our experiments, the embeddings are vectors, normalized to unit length, of dimension 384. (In larger sentence transformers the embedding dimension tends to be higher, e.g. 1024.) We used the sentence transformer in its pretrained state, i.e. we did not fine-tune it on our data. For further details on the model, see Sect. 4.7.

For each log type, Ladle extracts all extended log entries across the reference runs. Duplicate entries are eliminated but their counts are kept. The extended entries are embedded with the sentence transformer; these extended entries with their embeddings and counts constitute the reference data.

A typical sentence transformer has a context window of 128–512 tokens; the context window of our model of choice is 256. Most log entries in the EPP data consisted of less than 40 tokens; thus, with a sliding window length of $w = 4$ entries, 95.3% of extended entries fit within the context window. If the length, in tokens, of an extended entry exceeds the context window, Ladle discards the remaining tokens.

As every entry in a log gets a corresponding extended entry, the discarding of tokens never leads to a part of the log being omitted from the analysis unless an entry by itself exceeds the context window. This was the case for a mere 0.45% of

entries in the EPP data. However, for datasets where very long entries are more common, an alternative approach should be considered, such as using a sentence transformer with a longer context window or splitting long entries into chunks no longer than the context window.

When setting the sliding window length w, the following factors should be considered:

1. Ladle's ability to make use of a log entry's context depends on w being greater than one. In this sense, longer sliding windows are preferred.
2. The longer the sliding window, the coarser the anomaly localization resolution. As an extreme case, consider a sliding window as long as the anomalous log file itself. Then all extended entries up to the anomalous entry will contain the anomaly and will show heightened anomaly scores.
3. A sliding window consistently longer than the model's context window is meaningless. An extended entry can only be as long (measured in tokens) as the context window; extending the sliding window beyond that makes no difference.

From these considerations, we found $w = 4$ a suitable value for a sentence transformer with a context length of 256. We also tested the values $w = 3$ and $w = 5$ but found that they did not improve anomaly detection performance. However, when using a sentence transformer with a longer context window, a longer sliding window should be considered.

4.4 Anomaly score assignment

To assign an anomaly score to an extended log entry e in a log file in the test set, we first compare its embedding vector $\mathbf{v}_e$ to the embeddings of all extended log entries in the reference data for that log type. We take the mean distance to the k nearest embeddings:

$$\text{mean distance} = \frac{1}{k} \sum_{i=1}^{k} d(\mathbf{v}_e, \mathbf{r}_{ie}),$$

where $\mathbf{r}_{ie}$ is the reference vector that is i'th nearest to $\mathbf{v}_e$ and d is a distance metric, e.g. ℓ_2. The k nearest embeddings need not be unique; if the same extended entry appears m times in the reference set, the corresponding embedding counts as m embeddings.[4] For a schematic illustration of the calculation of the mean distance, see Fig. 6.

Ladle allows the user to choose the distance metric d; the supported options are Euclidean distance (ℓ_2), squared Euclidean distance, and cosine distance. In our experiments on the EPP data, all three options produced very similar results, with Euclidean distance slightly outperforming the other two.

[4] For performance reasons, only one copy of the embedding vector is kept in the database. The number of occurrences is stored separately.

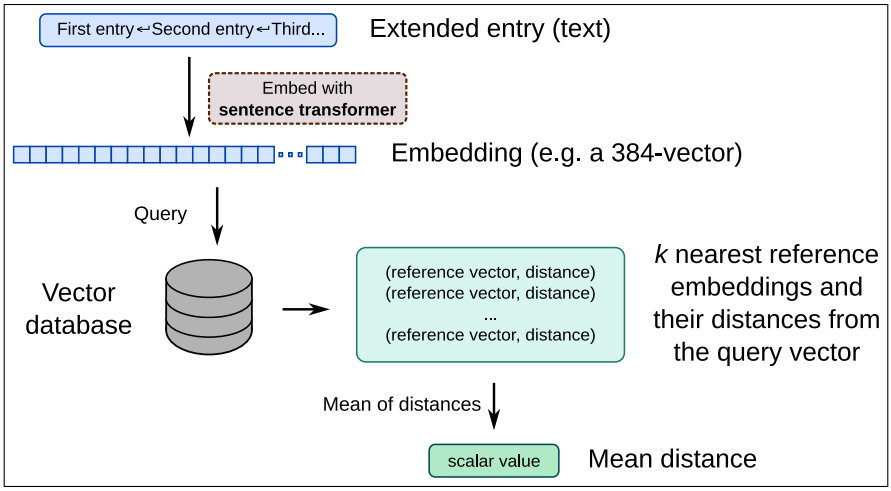


Fig. 6 The calculation of the mean distance of an extended entry embedding to the nearest reference embeddings

We then subtract from the mean distance a log-type-specific baseline score b to obtain the anomaly score for the extended entry:

$$\text{extended-entry anomaly score} = \text{mean distance} - b.$$

The purpose of the baseline score b is to make the anomaly scores comparable between log types; therefore, logs whose contents tend to vary little are assigned a small baseline score while more variable logs are assigned a larger baseline score. The baseline scores for the different log types are determined using the validation set. Their calculation is explained in detail in Sect. 4.5.

As each extended entry potentially contains information about the next $w - 1$ entries, where w is the sliding window length, we experimented with averaging the extended-entry anomaly scores over a window of length w. However, as the averaging did not improve anomaly detection accuracy, we omitted it in the experiments we report in this article.

The integer parameter k tells Ladle how many times a pattern needs to appear in the reference data to stop being novel. If, for a given embedding in the test data, at least k similar embeddings exist in the reference set, the anomaly score will be low. We used $k = 4$ in our experiments, but the value should be chosen to suit the application.

4.5 Baseline score

Some log types exhibit much variation between runs, while others remain nearly or exactly constant. Other things equal, a novel entry should be considered more indicative of an anomaly when it occurs in a less variable log type than when it occurs in a more variable log type. To that end, we use the validation set to determine how

much novelty is typically associated with each log type. The resulting value is the baseline score b. Its calculation for a given log type is presented in Algorithm 1.

Algorithm 1 Calculate Baseline

Require: *validation_entries*: DataFrame, *log_type*: String, $trim = 0.1$: Real
Ensure: Baseline value
log_entries ← rows in *validation_entries* where *validation_entries*[log_type] = *log_type*
runs ← unique values from *log_entries*[run]
max_scores ← empty array
for each *run* in *runs* **do**
    entries ← rows in *log_entries* where *log_entries*[run] = *run*
    Append max(*entries*[score]) to *max_scores*
end for
$trim \leftarrow \lfloor \text{length}(max\_scores) \times trim \rfloor$
baseline ← mean of sorted(*max_scores*) excluding *trim* smallest and largest values
return *baseline*

For every run in the validation set, an extended-entry anomaly score is calculated for every extended entry in the log file corresponding to the log type. (In Algorithm 1, these anomaly scores are assumed to be stored in the 'score' column of the dataframe.) The maximum anomaly score over extended entries becomes the anomaly score for that log file. Finally, b is set to the trimmed mean,[5] over validation runs, of these scores.

We chose the trimmed mean because it combines some of the strengths of the mean and the median, neither of which performed well by itself. Consider a hypothetical case where there are 20 validation runs. Suppose that the maximum anomaly score for some log type is $1.0 \cdot 10^{-22}$ (i.e. practically zero) in 11 runs, $1.0 \cdot 10^{-4}$ in 8 runs and $1.0 \cdot 10^{-1}$ in the remaining one. The median over these scores is $1.0 \cdot 10^{-22}$, which is not a good indicator of the variability in that log type. The mean, on the other hand, is $5.0 \cdot 10^{-3}$. This is not an entirely indefensible value, but it makes the score highly dependent on the chance event of the run with the high anomaly score being chosen into the validation set. However, if we use the trimmed mean, we get a reasonable score of $4.4 \cdot 10^{-5}$ and thus strike a good balance between sensitivity and robustness.

4.6 Updating ladle

To deal with data drift, Ladle should be updated with new data as necessary. This is done simply by adding new reference entries and their embeddings. When a significant amount of data drift has taken place, the baseline scores should also be recomputed.

An important aspect of Ladle that we relegate to future research is how updates to the reference and validation data should be scheduled. New logs can be added

[5] Specifically, we remove the top and bottom 10% and take the mean of the remaining values.

to the reference data as soon as they are judged normal, and if the new logs have gone through inference, the embeddings of their extended entries are already known. However, if new runs are continually added to the reference set without removing any, the vector databases will eventually grow large, increasing memory requirements and slowing down inference. We did not investigate how quickly such changes reach a meaningful magnitude, but when deploying Ladle to production, it is likely necessary to set up a mechanism for the automated removal of old entries.

Updates to the validation set, and recalculations of the baseline scores, should probably be performed less often than updates to the reference set. However, if new data are partitioned into the reference and validation sets so that the proportion of the reference set is very large (e.g. 90:10), data drift will be reflected in the baseline scores relatively slowly. This effect is partially counteracted by the fact that new entries will quickly accumulate in the reference data and cease to be judged as anomalous.

An interesting case is when a new log type appears after the initial deployment of Ladle. As it is desirable to obtain a value for the baseline score, it may be beneficial to initially assign new logs of a novel log type to the validation data at a higher rate than normal. As the baseline score can be calculated independently for each log type, it may be advantageous to update the baseline scores of relatively novel log types more often.

4.7 Implementation

We implemented Ladle in Python 3, using PyTorch as the neural network backend. We imported the sentence transformer (Reimers et al. 2019) LLMs (described in more detail below) from the Sentence Transformers library.[6] We used the Faiss (Douze et al. 2024; Johnson et al. 2021) library[7] for the embedding vector search in the interest of facilitating experimentation with various settings; however, the search could also be easily implemented using ordinary arrays.

We experimented with a few different models from the Sentence Transformers library. The overall performance of various sentence transformers, based on multiple benchmarks, can be studied at the MMTEB (Massive Multilingual Text Embedding Benchmark) leaderboard.[8] We initially performed most of our experiments with the model designated `all-MiniLM-L6-v2`, a very fast and relatively small model that nevertheless achieves a good ranking on the leaderboard. However, `avsolatorio/GIST-small-Embedding-v0`, another small model, produced slightly higher accuracy scores in our experiments. The accuracy scores reported in this article were obtained using the latter model.

We found that somewhat larger models, despite showing better scores on the leaderboard, did not significantly improve Ladle's anomaly detection accuracy. We

[6] https://sbert.net/.

[7] https://github.com/facebookresearch/faiss.

[8] https://huggingface.co/spaces/mteb/leaderboard.

did not experiment with significantly larger models as they were too slow for our purposes.

Some sentence transformers, like `all-MiniLM-L6-v2`, produce embeddings whose magnitude is normalized to unity. This is not the case with all sentence transformers; if a model with nonnormalized embeddings is selected, the choice of distance or similarity metric (ℓ_2, dot product, or cosine distance) for the vector search may affect the accuracy of the method.

5 Experiments

5.1 Dataset

We tested Ladle on a dataset of log files from an industry use case. The logs were produced by the test suite of an EPP. The dataset contained logs from 211 runs of the test suite, each of which produced an average of 170 log files (minimum 160, maximum 179). We treated each file, identified by its filename, as a log type. There were 224 log types in total, of which 127 log types (56.7%) were present in all runs. A histogram of the percentage of runs in which each log type was present can be seen in Fig. 7a.

All logs in the dataset come from test runs in which no test failures occurred. We therefore treated them as normal data. This is intended to reflect actual usage conditions: the reference and validation data are automatically collected when no errors are known to have occurred but are not checked by hand for anomalous entries.

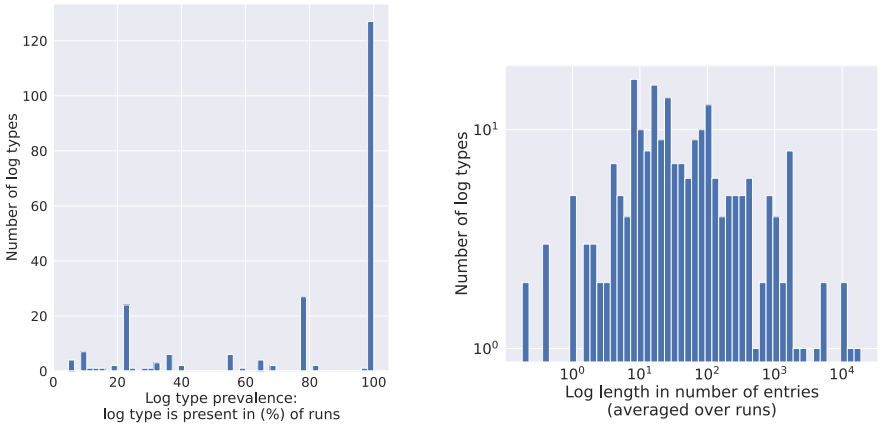
Log length, averaged over runs, varied significantly by log type. The median log type averaged 53 entries, while the longest log type averaged 18,637 entries.[9] A histogram of the average log lengths is presented in Fig. 7b.

5.2 Anomaly generation

Failures in the test suite of the EPP, and hence anomalies indicative of a failure, occur rarely. Although we were able to obtain some anomalous logs for manual inspection, they were not sufficient in number for a systematic evaluation of the accuracy of our method. We therefore tested Ladle on artificially generated anomalies.

We generated anomalies by introducing entries from a different log type. First, we randomly chose the target log type. In a random location in the target log file, we replaced a contiguous segment of a fixed number of entries with a segment of equal length (i.e. consisting of the same number of entries) from a randomly chosen different log type. We used four different anomaly sizes: 1, 2, 3, or 4, entries. We chose this anomaly generation method because it allows us to inject entries that occur in actual logs but are unexpected for the target log type. The generated

[9] The average log length for each log type was calculated over the runs in which the file was present.



(a) A histogram of the prevalences of the various log types, i.e. the share of runs in which each log type is present. Most log types are present in all runs.

(b) A histogram of log file lengths, averaged over runs. Log types with an average length of a few entries to a few hundred entries are the most common. A handful of log types have an average length of more than 10^4 entries.

Fig. 7 The prevalence of log types and average log lengths in the EPP data

anomalies therefore cannot be detected by looking for text that would be obviously out of place in the logs of a software product of this kind.

In reality, anomalies in logs are diverse and not limited to contiguous blocks of unexpected entries. The atypical entries may be spread out within a single log file or across multiple files. An anomaly might also consist of repeated log entries, each of which may by itself be innocuous; this would be the case, for example, when a process repeatedly attempts to fetch a file but fails due to a bad network connection. Some problems may manifest as the absence of log entries that would otherwise be present. However, these more subtle anomalies are difficult to simulate realistically without human involvement.

While falling short of naturally occurring anomalies in realism, the generated anomalies allow us to test the performance of Ladle against an existing anomaly detection method, as discussed in Sect. 5.4, on the multi-log-type task. The comparison is fair in that neither method is specifically engineered to detect anomalies of this kind.

5.3 Test setup and metrics

We calculated Ladle's top-1, top-5, and top-10 accuracy on the EPP log data for each anomaly size (1–4 entries). The top-N accuracy is the percentage of test iterations where the target log type (i.e. the log type containing the artificial anomaly) was within the first N log types when sorted by anomaly score.

In each test iteration, a single run randomly selected from the holdout set functioned as the test run. The test run can be thought of as the collection of log

files (representing various log types) that is about to undergo inference. From the log types present in the test run, we randomly selected a single log type as the target log type. After introducing an anomaly into the log corresponding to the target log type, we used Ladle to assign an anomaly score to every entry in each log file in the test run. We then ordered the log types according to the highest anomaly score assigned to any entry, recording the rank of the target log type.

Of the 211 runs in the EPP dataset, we randomly assigned 189 runs ($\approx 90\%$) to the reference set and the remaining 22 runs to the holdout set. In every test iteration, a single run was designated as the test run, leaving 21 runs as validation data. As the holdout set could be partitioned in 22 ways (by changing which run was used for testing), we performed an equal number of test iterations on each partitioning to produce more robust test results.

For each of the four anomaly sizes, we performed $20 \cdot 22 = 440$ test iterations, i.e., 20 iterations for each partitioning of the holdout set.

We did not separately measure anomaly localization accuracy; as our artificial anomalies are contiguous, any anomaly correctly detected by Ladle in our test setup will show the highest anomaly score no further than $w - 1$ entries from the anomaly.

5.4 Comparison with the state of the art in log anomaly detection

A method specifically created for anomaly detection across log types, like Ladle, is only needed if existing methods, designed for anomaly detection in a single stream of log outputs, cannot be easily adapted to perform the task with sufficient accuracy. To see whether this is the case, we tested the performance of a leading single-stream anomaly detection method, CAT (Zhang et al. 2022),[10] on the same multi-log-type task. The experiment was not intended to compare CAT's overall usefulness with Ladle's but to establish whether a separate method designed for the multi-log task is needed in the first place.

In the CAT method, logs are parsed according to templates generated from the data. Segments of fixed size (i.e. of a fixed number of log entries), extracted from normal (non-anomalous) logs and parsed, are used to train the neural network. We fitted the parser on the entire reference dataset and randomly selected 10000 segments for neural network training; we found that larger training subsets failed to improve accuracy but caused a sharp increase in training time. We modified the available code for CAT to directly output anomaly scores for test observations, which allowed us to rank log files by the maximum anomaly score assigned to any entry.

As CAT processes its inputs as log segments, we partitioned the entire test run into segments and fed them into the model sequentially. This is almost equivalent to concatenating the various log types and processing them as a single log. However, as we needed to be able to assign anomaly scores to log types, we disallowed log segments spanning two (or more) log types, instead terminating the log segments at file boundaries.

[10] As noted in Sect. 3, results surpassing those of CAT are reported for the MLAD method (Zang et al. 2024), but we were unable to obtain the code for the latter.

5.5 Adaptability to data drift

To test Ladle's ability to adapt to data drift, we obtained a small batch of EPP log data similar to the main dataset but produced at a different time. We will refer to the smaller dataset as the drift dataset. It consisted mostly of the same log types as the main dataset but with slightly different contents. We simulated two situations:

1. **Immediately after data drift occurs.** The main dataset was split into a reference set and a holdout set as before, but this time all runs in the holdout set were used as validation runs. A run from the drift dataset was brought in as the test run. This represents a situation where data drift has occurred but is not yet reflected in the reference and validation sets.
2. **Some time after data drift has occurred.** Nine runs from the drift dataset were added to the main dataset, and the resulting dataset was randomly split between the reference and validation sets. A run from the drift dataset was again used as the test run. This represents a situation where data drift has occurred some time ago and is starting to be reflected in the reference and validation sets.

We used the same run from the drift set as the test run in both scenarios. For each scenario, we performed 200 test iterations. In each test iteration, an anomaly was randomly introduced into a single log type in the test run in the same way as in the main experiment. We made two predictions for the outcome of the data drift experiment:

1. Anomaly detection accuracy in Scenario 1 should be lower than in the main experiment, as the reference set is not representative of the "new normal" after data drift.
2. Anomaly detection accuracy should rebound in Scenario 2, as extended entries reflecting the data drift have been incorporated into the reference set.

We excluded all log types in the drift set that were not present in the main dataset as including them would have exaggerated Ladle's data drift adaptation performance. From Ladle's point of view, an anomalous log is indistinguishable from a normal one if it is of a new log type. Therefore, if new log types were permitted, Scenario 2 might have seen an increase in accuracy compared to Scenario 1 simply due to the new log types having some reference entries rather than none.

6 Results

6.1 Anomaly detection accuracy

The results of our experiment are summarized in Table 1. The shortest anomalies (1 entry long) were the most difficult to detect, with a top-1 accuracy of 77.62%. The top-1 accuracy improved with anomaly size, rising to 89.05% for length 2, 93.81%

for length 3, and 97.86% for length 4. The top-5 accuracy was over 94.0% for all anomaly sizes, exceeding 99.0% for lengths 3 and 4. The top-5 accuracy was more than 97.0% for all anomaly sizes and 99.0% for all sizes greater than 1.

There were on average 169 log types (minimum 160, maximum 178, median 169) in each test iteration. The top-N positions therefore translate to the following percentages: top-10 corresponds to the top 5.92% of the compared log types, top-5 to 2.96%, and top-1 to 0.59%.

The top-1 and top-5 accuracies of CAT were below 30.0% for all anomaly sizes, and the top-10 accuracy was below 31.0% for all anomaly sizes. For Ladle, longer anomalies were easier to detect than shorter ones, but no such pattern was seen for CAT. A comparison of top-1 accuracies between Ladle and CAT is presented in Fig. 8.

6.2 Adaptation to data drift

Data drift caused a significant drop in accuracy before adaptation; the effect was particularly pronounced in the case of short anomalies. With adaptation, the accuracy scores recovered. The results for all anomaly sizes are presented in Fig. 9.

6.3 Significance of the baseline score

To test whether re-centering the raw anomaly scores by the baseline score b improves accuracy, we reran the experiments with re-centering disabled. Re-centering improved accuracy significantly especially for smaller anomalies: for anomalies of size one, the top-1 accuracy was 10.24 percentage points higher with re-centering (77.62% vs. 67.38%). The results for all anomaly sizes are shown in Fig. 10.

7 Discussion

Many systems for detecting and localizing anomalies in a single stream of log outputs exist. In principle, collections of logs consisting of multiple log types and exhibiting within-type homogeneity and between-group heterogeneity can be handled by such a system if the logs from the various log types are concatenated or interleaved into a single stream or are processed sequentially. However, as our experiment with the state-of-the-art method CAT indicates, this is in practice not a promising approach with existing methods.

By contrast, Ladle performs very well on the multi-log-type anomaly detection task. Our results show that a part of Ladle's advantage over single-stream log anomaly detection methods on the multi-log-type problem is due to the re-centering of anomaly scores based on how much variation is known to occur in each log type. However, even without re-centering, Ladle clearly outperforms the alternative. This suggests that an approach which, like Ladle, separates the task of learning the

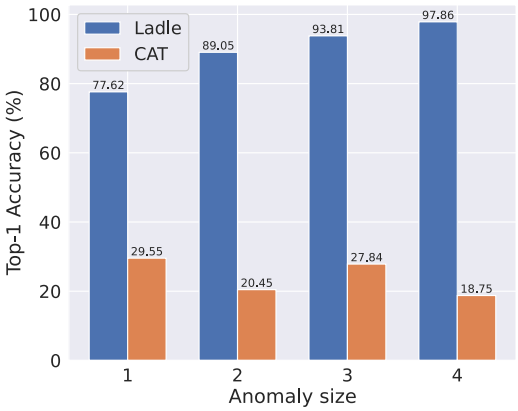


Fig. 8 The top-1 accuracies of Ladle (our method) and CAT by anomaly size

Table 1 Accuracy by anomaly size: a comparison between Ladle and CAT, a leading method for unsupervised single-stream log anomaly detection

	Anomaly Size = 1		Anomaly Size = 2	
	Ladle	CAT	Ladle	CAT
Top-1	**77.62%**	29.55%	**89.05%**	20.45%
Top-5	**94.52%**	29.55%	**98.10%**	20.45%
Top-10	**97.62%**	30.11%	**99.76%**	21.02%
	Anomaly Size = 3		Anomaly Size = 4	
	Ladle	CAT	Ladle	CAT
Top-1	**93.81%**	27.84%	**97.86%**	18.75%
Top-5	**99.29%**	29.55%	**99.52%**	22.16%
Top-10	**99.52%**	30.11%	**99.76%**	23.86%

For each metric, the higher (better) value is indicated in bold

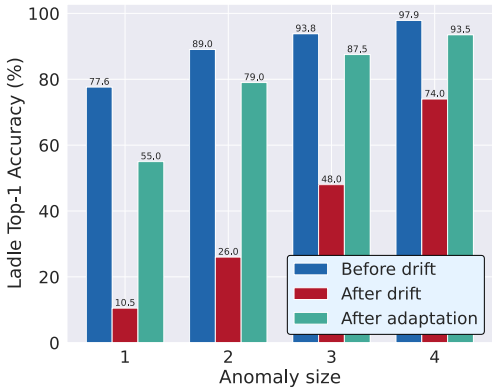


Fig. 9 The effects of data drift before and after adaptation

specifics of the log data from the task of learning natural language, could be viable in the single-log-stream domain as well.

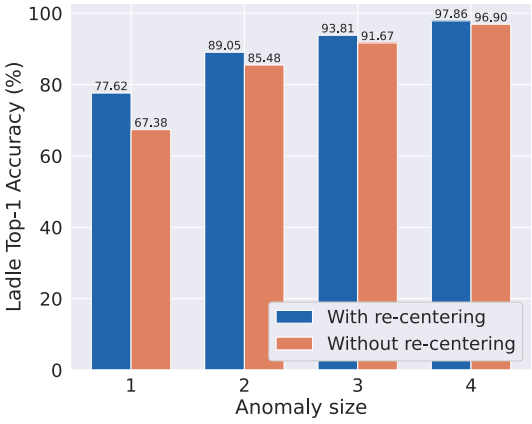


Fig. 10 The effect of re-centering the anomaly scores

There is currently a dearth of software tools for fast experimentation with different log anomaly detection methods. An emerging Python tool that may meet this need in the future is LogLead (Mäntylä et al. 2024). LogLead allows the user to apply different supervised or unsupervised anomaly detection techniques to a number of publicly available datasets or to a user-defined dataset but does not yet explicitly support anomaly detection across log types.

Our testing procedure for Ladle relies on anomalies generated by replacing log entries with entries from another log type. Many real-life anomalies cannot be accurately reproduced by this procedure. Better and more diverse methods for log anomaly generation would facilitate the evaluation of anomaly detection and localization systems. In the future, LLMs may be able to generate realistic artificial anomalies that would otherwise be difficult to produce without human intervention.

8 Conclusions, limitations and future work

We propose Ladle, an accurate tool for detecting and localizing anomalies in log data consisting of many log types and exhibiting within-group homogeneity and between-group heterogeneity. We demonstrate the accuracy of Ladle using a real-world log dataset. We also show that the existing state of the art in single-log-type anomaly detection does not meet the need for anomaly detection in this kind of setting.

Our method can accommodate data drift and has an explicit human-interpretable parameter to control how quickly it adapts to new features in normal data. Ladle is updated by adding new data with no need for neural network retraining.

A limitation of our research is that we can currently only test our method on artificial anomalies due to the lack of applicable datasets. While existing datasets with known anomalies could be artificially combined by treating each dataset as a different log type, the resulting dataset would lack a crucial feature of real-world data, namely that a single cause (e.g. a change in network connectivity) can affect multiple log types.

The design choices we experimented with in this research were not very consequential with regard to the trade-off between anomaly detection performance and anomaly localization performance. However, such trade-offs will likely present themselves in the future due to, e.g., sentence transformers with longer context windows becoming more prevalent. To make informed choices about these trade-offs it will be necessary to measure not only the anomaly detection accuracy of the method, as we have done here, but also the anomaly localization performance, for which the AUROC score is typically used.

As discussed in Sect. 4.6, more research is required to determine how updates to the reference and validation datasets should be scheduled to maintain a good balance between computation, storage and memory requirements, and responsiveness to data drift.

Acknowledgements The authors wish to acknowledge CSC - IT Center for Science, Finland, for computational resources.

Author Contributions J.M. formulated, implemented, and tested the method and wrote the manuscript text. T.A. procured the data and provided insight into industry needs. J.K.N. supervised the project.

Funding Open Access funding provided by University of Helsinki (including Helsinki University Central Hospital).

Data Availability Ladle is publicly available on GitHub https://github.com/juhamyllari/ladle. The EPP log dataset was used with permission from WithSecure. It is not publicly available due to confidentiality agreements.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

Devlin, J., Chang, MW., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. https://doi.org/10.48550/arXiv.1810.04805, arXiv:1810.04805 [cs] (2019)

Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, PE., Lomeli, M., Hosseini, L., Jégou, H.: The Faiss library. https://doi.org/10.48550/arXiv.2401.08281, arXiv:2401.08281 [cs] (2024)

Du, M., Li, F., Zheng, G., Srikumar, V.: DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security, association for computing machinery, New York, NY, USA, CCS '17, pp 1285–1298, https://doi.org/10.1145/3133956.3134015 (2017)

Guo, H., Yuan, S., Wu, X.: LogBERT: Log anomaly detection via BERT. In: 2021 international joint conference on neural networks (IJCNN), pp 1–8, https://doi.org/10.1109/IJCNN52387.2021.9534113, iSSN: 2161-4407 (2021)

He, S., Zhu, J., He, P., Lyu, MR.: Experience Report: System log analysis for anomaly detection. In: 2016 IEEE 27th international symposium on software reliability engineering (ISSRE), pp 207–218, https://doi.org/10.1109/ISSRE.2016.21, https://ieeexplore.ieee.org/abstract/document/7774521, iSSN: 2332-6549 (2016)

Johnson, J., Douze, M., Jégou, H.: Billion-Scale Similarity Search with GPUs. IEEE Transact. Big Data **7**(3), 535–547 (2021). https://doi.org/10.1109/TBDATA.2019.2921572

Le, VH., Zhang, H.: Log-based anomaly detection without log parsing. In: 2021 36th IEEE/ACM international conference on automated software engineering (ASE), pp 492–504, https://doi.org/10.1109/ASE51524.2021.9678773, https://ieeexplore.ieee.org/abstract/document/9678773, iSSN: 2643-1572 (2021)

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih Wt, Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. arXiv:2005.11401 [cs] (2021)

Mäntylä, M., Wang, Y., Nyyssölä, J.: LogLead – Fast and integrated log loader, enhancer, and anomaly detector. https://doi.org/10.48550/arXiv.2311.11809, arXiv:2311.11809 [cs] (2024)

Pan, J., Wong, SL., Yuan, Y.: RAGLog: Log anomaly detection using retrieval augmented generation. https://doi.org/10.48550/arXiv.2311.05261, arXiv:2311.05261 [cs] (2023)

Reimers, N., Gurevych, I.: Sentence-BERT: Sentence embeddings using siamese BERT-networks. https://doi.org/10.48550/arXiv.1908.10084, arXiv:1908.10084 [cs] (2019)

Yadav, RB., Kumar, PS., Dhavale, SV.: A survey on log anomaly detection using deep learning. In: 2020 8th international conference on reliability, infocom technologies and optimization (Trends and Future Directions) (ICRITO), pp 1215–1220, https://doi.org/10.1109/ICRITO48877.2020.9197818, https://ieeexplore.ieee.org/abstract/document/9197818 (2020)

Zang, R., Guo, H., Yang, J., Liu, J., Li, Z., Zheng, T., Shi, X., Zheng, L., Zhang, B.: MLAD: A unified model for multi-system log anomaly detection. https://doi.org/10.48550/arXiv.2401.07655, http://arxiv.org/abs/2401.07655, arXiv:2401.07655 [cs] (2024)

Zhang, S., Liu, Y., Zhang, X., Cheng, W., Chen, H., Xiong, H.: CAT: Beyond efficient transformer for content-aware anomaly detection in event sequences. In: proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining, association for computing machinery, New York, NY, USA, KDD '22, pp 4541–4550, https://doi.org/10.1145/3534678.3539155 (2022)

Zhu, J., He, S., He, P., Liu, J., Lyu, MR.: Loghub: a large collection of system log datasets for AI-driven log analytics. In: 2023 IEEE 34th international symposium on software reliability engineering (ISSRE), pp 355–366, https://doi.org/10.1109/ISSRE59848.2023.00071, https://ieeexplore.ieee.org/abstract/document/10301257, iSSN: 2332-6549 (2023)